

Project Descriptions (Brief)

Britt Anderson

Feb 25 2026

Contents

1	Overview	1
2	Evaluation	2
3	Project Topics	3
3.1	Reinforcement Learning	3
3.1.1	Starting References and Resources	3
3.2	Self-organizing Maps (Kohonen Neural Networks)	4
3.2.1	Starting References and Resources	4
3.3	Morris-Lecar Networks	4
3.3.1	Starting References and Resources	4
3.4	Backpropagation	5
3.4.1	Starting References and Resources	5
3.5	Agent Based Models	5
3.5.1	Starting References and Resources	6
3.6	Linear Ballistic Accumulators	6
3.6.1	Starting References and Resources	6
4	Project Deliverables Summarized	7
4.1	Presentation	7
4.2	Report	7
5	General Suggestions	7

1 Overview

The goals of the final project are:

1. Active learning
2. Learning through teaching
3. Acquiring important professional skills such as:
 - Presentation development
 - Oral presentation practice
 - Project Planning
 - Group collaboration
 - Technical Writing
 - Coding Relevant to Psychology and Neuroscience Modeling
 - Technical Knowledge

In pursuit of these goals a fair amount of time in the last few weeks will be saved for the group project. Each of you will have been assigned by me to a group. The members of the groups will work on the project collaboratively dividing the responsibilities and workload as jointly decided.

The deliverables will be two. First, there will be on the last course day a 1/2 hour presentation that will:

1. Provide a general introduction to the topic. This will not only be a technical introduction, but will help us understand the history and motivation for using this tool/technique.
2. A tutorial survey of the details. This might include the mathematical underpinnings and derivation as well as any pertinent novel notation.
3. Practical aspects of coding the algorithm. Ideally working code written by the group will be demonstrated. This is expected to be simple and modest in scope. So, it might be appropriate in addition to the group's own code to show off libraries or software written by others and useful for more complicated or realistic demonstrations.

The emphasis of the oral presentation is on learning. Both of us: the audience and the group presenting. This is a time to practice your presentation style and any novel ideas you have about how to give a memorable and instructive presentation. There will be no grade beyond a complete/incomplete.

The goal of the written reports is to provide an opportunity to develop and share a more rigorous and complete idea of the same general scope, but

with increased detail and space for explanations. While a written document is expected other materials may make sense as part of the submission. For example, source code is likely to be a part of all projects. In addition, images or movies of simulations and similar materials may be appropriate. That is for the group to decide.

2 Evaluation

1. For the oral presentation the evaluation is strictly limited to completion. However, I hope that students will look upon this as an opportunity for practice, and also will be respectful of their peers time. You will be listening much more than speaking so keep that in mind as you plan the quality of your own offering.
2. For the written, final presentation I will be evaluating:
 - Clarity, completeness, and professionalism of the writing.
 - Documentation of the topic should include appropriate citations. I don't care so much about what citation style you use as long as it is consistent and follows some professional standard.
 - The tutorial code provided will be evaluated for style, accuracy, and the ease with which someone else (me for example) can get it working properly. That may mean the inclusion of instructions.
 - The value of any additional materials.
 - A general assessment of the pedagogical value of the presentation as a guide to help others understand the material. Ask yourself if your presentation would be a good guide for me as an instructor to use it as the basis for developing additional units of material to present in this course?

3 Project Topics

Each group will be assigned one of the topics below. Send me your groups ordered preference list as soon as possible. I have added some starter references, but those should not be viewed as sufficient in and of themselves. It is just to give a chance to get a better sense of what the topic is about and what kind of resources you might be looking for.

3.1 Reinforcement Learning

There is a "classic" textbook available on line, code as well. Reinforcement Learning is a common paradigm and mechanism for explaining a multitude of behaviors. The basic idea is that you do more of whatever it is that gives you a reward. The mapping of dopamine dynamics onto reward delivery has been heralded as one of the great successes of computational neuroscience.

3.1.1 Starting References and Resources

- The canonical textbook, freely available online, is Sutton & Barto's *Reinforcement Learning: An Introduction* (2nd ed.). Both the PDF and code examples are available at: <http://incompleteideas.net/book/the-book-2nd.html>
- For the neuroscience connection, see the dopamine and reward prediction error review cited below.
- A practical Python-focused tutorial series using OpenAI Gym (now Gymnasium) is a good hands-on complement: <https://gymnasium.farama.org/>

3.2 Self-organizing Maps (Kohonen Neural Networks)

Ever wonder how we develop those remarkable maps in the brain where there seems to be an organizational principle that puts similar things next to each other? There is an R package for Kohonen maps, and an article that describes it. For more on the idea itself there are many treatments. One possible starting point would be Kohonen himself.

3.2.1 Starting References and Resources

- Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1), 59–69. <https://doi.org/10.1007/BF00337288> — the original paper.
- Wehrens, R., & Buydens, L. M. C. (2007). Self- and super-organizing maps in R: The kohonen package. *Journal of Statistical Software*, 21(5). <https://doi.org/10.18637/jss.v021.i05> — the primary reference for the R kohonen package.

- The R `kohonen` package documentation and vignettes are a practical starting point for coding: <https://cran.r-project.org/package=kohonen>

3.3 Morris-Lecar Networks

In order to gain insights into the dynamics of models of neurons it is useful to simplify them from their fully biologically realistic instances. One simplified version is the Morris-Lecar model that has fewer moving parts, but yields rich dynamics. An introduction from a textbook is available on line. Code also can be found all over the internet, and you should have code, but this might also be a good chance to look at this tool widely used by the neural dynamics community. The author of the software also has his own textbook available for download.

3.3.1 Starting References and Resources

- Morris, C., & Lecar, H. (1981). Voltage oscillations in the barnacle giant muscle fiber. *Biophysical Journal*, 35(1), 193–213. [https://doi.org/10.1016/S0006-3495\(81\)84782-0](https://doi.org/10.1016/S0006-3495(81)84782-0) — the original paper.
- Rinzel, J., & Ermentrout, G. B. (1998). Analysis of neural excitability and oscillations. In C. Koch & I. Segev (Eds.), *Methods in Neuronal Modeling* (2nd ed., pp. 251–291). MIT Press. — a classic chapter situating Morris-Lecar in the broader landscape of neural dynamics. (I found an online copy [here](#))
- Ermentrout, B., & Terman, D. (2010). *Mathematical Foundations of Neuroscience*. Springer. — Ermentrout’s textbook is freely downloadable and treats Morris-Lecar in depth; his simulation tool XPPAUT is also widely used: <http://www.math.pitt.edu/~bard/xpp/xpp.html> or <https://sites.pitt.edu/~phase/bard/bardware/xpp/xpp.html>.
- The Brian2 simulator provides a Python environment well suited to implementing Morris-Lecar networks: <https://brian2.readthedocs.io/>

3.4 Backpropagation

This is the method that revolutionized the world of neural networks. It was a means of transmitting the error to each unit in a multilayer perceptron that would allow for effective quick learning of complex problems. The derivation

of the method is complicated in the details, but relatively straightforward in the broad strokes. There are many, many demonstrations of this, but writing your own code to implement the method teaches you a lot.

3.4.1 Starting References and Resources

- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. <https://doi.org/10.1038/323533a0> — the landmark paper.
- Nielsen, M. (2015). *Neural Networks and Deep Learning*. Free online book with worked examples and code in Python: <http://neuralnetworksanddeeplearning.com/> — an excellent and readable tutorial treatment of backpropagation from first principles.
- 3Blue1Brown’s video series “Neural Networks” on YouTube provides outstanding visual intuition for the algorithm before diving into the mathematics: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

3.5 Agent Based Models

Agent Based Models (ABMs) allow us to simulate how complex, population-level patterns emerge from the local interactions of many individual agents, each following simple rules. They have been applied to problems ranging from the spread of disease to the emergence of social norms and collective neural dynamics.

3.5.1 Starting References and Resources

- Axelrod, R. (1997). *The Complexity of Cooperation: Agent-Based Models of Competition and Collaboration*. Princeton University Press. — a foundational text on ABMs in the social and behavioral sciences.
- NetLogo is a widely used, beginner-friendly ABM environment with an extensive model library and its own tutorial: <https://ccl.northwestern.edu/netlogo/>
- Mesa is the primary Python library for agent-based modeling and a natural choice if you prefer to stay in Python. Documentation and tutorials are at: <https://mesa.readthedocs.io/> and the repository is at: <https://github.com/projectmesa/mesa>

- The Santa Fe Institute’s Complexity Explorer offers free online courses and resources that provide good conceptual grounding: <https://www.complexityexplorer.org/>

3.6 Linear Ballistic Accumulators

How do we make decisions? We accumulate evidence. That is the idea behind drift diffusion models. And they have been very big over the years, however the math is quite complex and they can be hard to fit. Linear ballistic accumulators are a simplified version. And just as we saw with integrate and fire models simple can sometimes be better, depending on your question.

3.6.1 Starting References and Resources

- Brown, S. D., & Heathcote, A. (2008). The simplest complete model of choice response time: Linear ballistic accumulation. *Cognitive Psychology*, 57(3), 153–178. <https://doi.org/10.1016/j.cogpsych.2007.12.002> — the paper that introduced the LBA model.
- Donkin, C., Brown, S. D., & Heathcote, A. (2011). Drawing conclusions from choice response time models: A tutorial using the linear ballistic accumulator. *Journal of Mathematical Psychology*, 55(2), 140–151. <https://doi.org/10.1016/j.jmp.2010.10.001> — a tutorial-style companion paper aimed at new users.
- The `rtdists` R package implements the LBA and related evidence-accumulation models and is a practical starting point for coding: <https://cran.r-project.org/package=rtdists>
- PyDDM is a Python-based simulator for drift-diffusion and accumulator models that can serve as a useful reference implementation: <https://pyddm.readthedocs.io/>

4 Project Deliverables Summarized

4.1 Presentation

A half hour presentation during the last class session that will provide us all with an overview of your topic to include its background, importance, and application. Then you will need to walk us through how to implement and use your computational offering. Lastly, you should address how it is

or could be used for better understanding mental or neural questions about the mechanisms of cognition.

4.2 Report

Your report should be a paper replete with the usual abstract, intro, methods, code, results of simulations, and references. Figures and analyses should be integrated into the manuscript in order to have a reproducible research project. To the extent that this is not felt to be possible you will have to justify it heavily. Tools that you might find useful include Overleaf and Quarto, though you can probably get close with Rmd/Rstudio since that now supports python. If you elect to use another language you may find they have a suitable language for this (e.g. with Racket it is Scribble).

5 General Suggestions

I will make the groups, and I encourage each of you to find the right role for each member to make the project work efficiently. Some may be stronger writers while others are better coders. All research is collaborative with a mix of talents. Finding out how to discover that, and apportion work will be important in your future endeavors.

We will decide together the deadline for the delivery of the final report.